Data Science and Analytics

Lab Course CS-308-MJ-P | T.Y.B.Sc.(C.S.) NEP-2020 Sem V

Practical Slip Solutions
Slips: 4, 5, 6, 7, 11, 14, 15, 16, 17, 18, 19, 20, 22, 23, 25

- Each slip lists Q1, Q2 and the Q2 OR option with complete Python code (no comments in the code). The Viva (Q3) is oral and is not included.

- These programs use only pandas, numpy and matplotlib (plus matplotlib-venn for Venn diagrams). scikit-learn, scipy, mlxtend, squarify and wordcloud are NOT used. Train-test split, scaling, KNN, K-Means, Logistic and Linear Regression, Random Forest, Apriori, treemap, dendrogram and word cloud are written with plain numpy / pandas / matplotlib code.

- Place the dataset CSV files in the same folder as the program, using the file names from the reference links in the question paper.

- If a dataset column name in your CSV differs from the one in the code, change it in the code (see the note under that question).

Running the Programs from the Terminal

1. Install the libraries (run once)

```
pip install pandas numpy matplotlib matplotlib-venn
```

Linux / Mac: use pip3

```
pip3 install pandas numpy matplotlib matplotlib-venn
```

2. Check that the libraries are installed

```
python -c "import pandas, numpy, matplotlib, matplotlib_venn"
```

3. Go to your practical folder (keep the CSV files here)

```
cd Desktop\DSA_Practical
```

4. Save each program in a file, for example slip4_q1.py, and run it

```
python slip4_q1.py
```

Linux / Mac

```
python3 slip4_q1.py
```

- matplotlib-venn is installed with that name but imported as matplotlib_venn. It is needed only for the Venn diagram programs (slips 5, 20, 22).

- Plot windows open when plt.show() runs; close the window to let the program continue to the next plot.

- If pip is not recognised, use: python -m pip install <library name>

Slip 4

Q1 [10 Marks]

Survey of preferred payment methods: payment_methods = [UPI, Credit Card, Debit Card, Cash, Net Banking], customers = [150, 80, 60, 40, 30]. Create a Pie Chart showing the percentage of customers for each method, display the percentage contribution and add a title.

```
import matplotlib.pyplot as plt

payment_methods = ["UPI", "Credit Card", "Debit Card", "Cash", "Net Banking"]
customers = [150, 80, 60, 40, 30]
colors = ["green", "blue", "orange", "pink", "purple"]

plt.pie(customers, labels=payment_methods, colors=colors, autopct="%1.1f%%")
plt.title("Preferred Payment Method of Customers")
plt.show()
```

Q2 [20 Marks]

Apply Logistic Regression on the Online Gaming dataset to classify players as Casual or Professional using daily play time, achievements unlocked, in-game purchases and gaming level. Evaluate with Accuracy, Precision, Recall, F1-Score and Confusion Matrix.

```
import numpy as np
import pandas as pd

df = pd.read_csv("online_gaming_behavior_dataset.csv")
df = df.dropna()

df["Player_Type"] = (df["EngagementLevel"] == "High").astype(int)

X = df[["PlayTimeHours", "AchievementsUnlocked", "InGamePurchases", "PlayerLevel"]].values.astype(float)
y = df["Player_Type"].values

np.random.seed(42)
index = np.random.permutation(len(X))
split = int(0.8 * len(X))
X_train, X_test = X[index[:split]], X[index[split:]]
y_train, y_test = y[index[:split]], y[index[split:]]

mean = X_train.mean(axis=0)
std = X_train.std(axis=0) + 1e-9
X_train = (X_train - mean) / std
X_test = (X_test - mean) / std

w = np.zeros(X_train.shape[1])
b = 0
for i in range(1000):
    p = 1 / (1 + np.exp(-(X_train @ w + b)))
    w = w - 0.1 * (X_train.T @ (p - y_train)) / len(y_train)
    b = b - 0.1 * np.mean(p - y_train)

y_pred = (1 / (1 + np.exp(-(X_test @ w + b))) >= 0.5).astype(int)

tp = np.sum((y_test == 1) & (y_pred == 1))
tn = np.sum((y_test == 0) & (y_pred == 0))
fp = np.sum((y_test == 0) & (y_pred == 1))
fn = np.sum((y_test == 1) & (y_pred == 0))

accuracy = (tp + tn) / len(y_test)
precision = tp / (tp + fp)
recall = tp / (tp + fn)
f1 = 2 * precision * recall / (precision + recall)

print("Accuracy :", accuracy)
print("Precision:", precision)
print("Recall   :", recall)
print("F1-Score :", f1)
print("Confusion Matrix:")
print(np.array([[tn, fp], [fn, tp]]))
```

Note: The dataset has no Casual/Professional column, so EngagementLevel = High is taken as Professional (1) and the rest as Casual (0). Logistic regression is trained with gradient descent in numpy.

Q2 OR [20 Marks]

Apply K-Means Clustering on the E-commerce Customer dataset. Read, inspect and preprocess the data (missing values, numerical features), cluster the customers and interpret the clusters.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

def kmeans(X, k, iterations=100):
    np.random.seed(42)
    centers = X[np.random.choice(len(X), k, replace=False)]
    for i in range(iterations):
        distances = np.sqrt(((X[:, None, :] - centers[None, :, :]) ** 2).sum(axis=2))
        labels = distances.argmin(axis=1)
        new_centers = np.array([X[labels == j].mean(axis=0) if np.any(labels == j)
                                else centers[j] for j in range(k)])
        if np.allclose(new_centers, centers):
            break
        centers = new_centers
    return labels, centers


df = pd.read_csv("Ecommerce Customers.csv")
print(df.info())
print(df.isnull().sum())
df = df.dropna()

features = ["Avg. Session Length", "Time on App", "Time on Website",
            "Length of Membership", "Yearly Amount Spent"]
X = df[features].values.astype(float)
X = (X - X.mean(axis=0)) / X.std(axis=0)

labels, centers = kmeans(X, 3)
df["Cluster"] = labels

print(df["Cluster"].value_counts())
print(df.groupby("Cluster")[features].mean())

plt.scatter(X[:, 3], X[:, 4], c=labels)
plt.scatter(centers[:, 3], centers[:, 4], color="red", marker="X", s=200, label="Centroids")
plt.xlabel("Length of Membership")
plt.ylabel("Yearly Amount Spent")
plt.title("Customer Clusters")
plt.legend()
plt.show()
```

Note: Interpretation: compare the cluster means printed by groupby. A cluster with high Length of Membership and high Yearly Amount Spent is the loyal / high-value segment, the one with low values on both is the low-value segment, and the middle one is the average segment.

Slip 5

Q1 [10 Marks]

Create a customer_DataFrame with Customer_ID, Gender, City and Membership_Type. Apply label encoding on the categorical attributes and display the DataFrame before and after encoding.

```
import pandas as pd

customer_DataFrame = pd.DataFrame({
    "Customer_ID": [1, 2, 3, 4, 5, 6],
    "Gender": ["Male", "Female", "Female", "Male", "Female", "Male"],
    "City": ["Nashik", "Pune", "Mumbai", "Nashik", "Pune", "Mumbai"],
    "Membership_Type": ["Gold", "Silver", "Bronze", "Gold", "Bronze", "Silver"]
})
print("Before encoding:")
print(customer_DataFrame)

for col in ["Gender", "City", "Membership_Type"]:
    customer_DataFrame[col] = customer_DataFrame[col].astype("category").cat.codes

print("After encoding:")
print(customer_DataFrame)
```

Q2 [20 Marks]

Generate a DataFrame of Netflix and Amazon Prime customers. Create a Venn diagram and a Pie chart showing customers using only Netflix, only Amazon Prime and both, with labels, colors and titles.

```
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib_venn import venn2

df = pd.DataFrame({
    "Customer": ["Amit", "Neha", "Rohan", "Sara", "Karan", "Priya", "Vikas", "Anita"],
    "Netflix": [1, 1, 0, 1, 0, 1, 1, 0],
    "Prime": [0, 1, 1, 1, 1, 0, 1, 1]
})
print(df)

netflix = set(df[df["Netflix"] == 1]["Customer"])
prime = set(df[df["Prime"] == 1]["Customer"])

only_netflix = len(netflix - prime)
only_prime = len(prime - netflix)
both = len(netflix & prime)

venn2([netflix, prime], set_labels=("Netflix", "Amazon Prime"), set_colors=("red", "blue"))
plt.title("Netflix and Amazon Prime Customers")
plt.show()

plt.pie([only_netflix, only_prime, both], labels=["Only Netflix", "Only Prime", "Both"],
        colors=["red", "blue", "yellow"], autopct="%1.1f%%")
plt.title("Netflix and Amazon Prime Customers")
plt.show()
```

Q2 OR [20 Marks]

Use the KNN algorithm to classify emails as spam or non-spam using the Spambase dataset. Preprocess, train and evaluate with accuracy, confusion matrix, precision, recall and F1-score.

```
import numpy as np
import pandas as pd

def knn_predict(X_train, y_train, X_test, k=5):
    predictions = []
    for row in X_test:
        distances = np.sqrt(((X_train - row) ** 2).sum(axis=1))
        nearest = y_train[np.argsort(distances)[:k]]
        values, counts = np.unique(nearest, return_counts=True)
        predictions.append(values[counts.argmax()])
    return np.array(predictions)


df = pd.read_csv("spambase.csv")
df = df.dropna()
```

```
X = df.iloc[:, :-1].values.astype(float)
y = df.iloc[:, -1].values.astype(int)

np.random.seed(42)
index = np.random.permutation(len(X))
split = int(0.8 * len(X))
X_train, X_test = X[index[:split]], X[index[split:]]
y_train, y_test = y[index[:split]], y[index[split:]]

mean = X_train.mean(axis=0)
std = X_train.std(axis=0) + 1e-9
X_train = (X_train - mean) / std
X_test = (X_test - mean) / std

y_pred = knn_predict(X_train, y_train, X_test, k=5)

tp = np.sum((y_test == 1) & (y_pred == 1))
tn = np.sum((y_test == 0) & (y_pred == 0))
fp = np.sum((y_test == 0) & (y_pred == 1))
fn = np.sum((y_test == 1) & (y_pred == 0))

accuracy = (tp + tn) / len(y_test)
precision = tp / (tp + fp)
recall = tp / (tp + fn)
f1 = 2 * precision * recall / (precision + recall)

print("Accuracy :", accuracy)
print("Precision:", precision)
print("Recall   :", recall)
print("F1-Score :", f1)
print("Confusion Matrix:")
print(np.array([[tn, fp], [fn, tp]]))
```

Note: The last column of spambase.csv is the spam label (1 = spam, 0 = non-spam).

Slip 6

Q1 [10 Marks]

Create a DataFrame with car model, engine size (litres) and fuel efficiency (km/L). Create a scatter plot of engine size vs fuel efficiency with a title, both axes labelled and different colors for the points.

```
import pandas as pd
import matplotlib.pyplot as plt

cars = pd.DataFrame({
    "Car_Model": ["Swift", "Baleno", "City", "Creta", "Fortuner", "Alto"],
    "Engine_Size": [1.2, 1.2, 1.5, 1.5, 2.8, 0.8],
    "Fuel_Efficiency": [22.4, 22.9, 17.8, 16.8, 10.0, 24.0]
})
print(cars)

colors = ["red", "green", "blue", "orange", "purple", "brown"]

plt.scatter(cars["Engine_Size"], cars["Fuel_Efficiency"], c=colors, s=100)
plt.title("Engine Size vs Fuel Efficiency")
plt.xlabel("Engine Size (litres)")
plt.ylabel("Fuel Efficiency (km/L)")
plt.show()
```

Q2 [20 Marks]

Use the Sports Performance dataset to build a Simple Linear Regression model predicting Performance Score from Training Hours Per Week. Evaluate with R2 score and plot a scatter graph with the regression line.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("collegiate_athlete_injury_dataset.csv")
print(df.columns)
df = df.dropna(subset=["Training_Hours_Per_Week", "Performance_Score"])

X = df["Training_Hours_Per_Week"].values
y = df["Performance_Score"].values

np.random.seed(42)
index = np.random.permutation(len(X))
split = int(0.8 * len(X))
X_train, X_test = X[index[:split]], X[index[split:]]
y_train, y_test = y[index[:split]], y[index[split:]]

slope, intercept = np.polyfit(X_train, y_train, 1)
y_pred = slope * X_test + intercept

ss_res = np.sum((y_test - y_pred) ** 2)
ss_tot = np.sum((y_test - y_test.mean()) ** 2)
print("Slope    :", slope)
print("Intercept:", intercept)
print("R2 Score :", 1 - ss_res / ss_tot)

line_x = np.sort(X_test)
plt.scatter(X_test, y_test, color="blue", label="Actual")
plt.plot(line_x, slope * line_x + intercept, color="red", label="Regression Line")
plt.xlabel("Training Hours Per Week")
plt.ylabel("Performance Score")
plt.title("Training Hours vs Performance Score")
plt.legend()
plt.show()
```

Note: Linear regression uses np.polyfit. Check the column names printed by df.columns and change them in the code if your file uses different names.

Q2 OR [20 Marks]

Implement a KNN classifier on the Customer dataset. Preprocess the data and classify customers into segments. Evaluate with Accuracy and Confusion Matrix.

```
import numpy as np
import pandas as pd

def kmeans(X, k, iterations=100):
```

```
    np.random.seed(42)
    centers = X[np.random.choice(len(X), k, replace=False)]
    for i in range(iterations):
        distances = np.sqrt(((X[:, None, :] - centers[None, :, :]) ** 2).sum(axis=2))
        labels = distances.argmin(axis=1)
        new_centers = np.array([X[labels == j].mean(axis=0) if np.any(labels == j)
                                else centers[j] for j in range(k)])
        if np.allclose(new_centers, centers):
            break
        centers = new_centers
    return labels, centers


def knn_predict(X_train, y_train, X_test, k=5):
    predictions = []
    for row in X_test:
        distances = np.sqrt(((X_train - row) ** 2).sum(axis=1))
        nearest = y_train[np.argsort(distances)[:k]]
        values, counts = np.unique(nearest, return_counts=True)
        predictions.append(values[counts.argmax()])
    return np.array(predictions)


df = pd.read_csv("Mall_Customers.csv")
df = df.dropna()
df["Genre"] = df["Genre"].astype("category").cat.codes

X = df[["Genre", "Age", "Annual Income (k$)", "Spending Score (1-100)"]].values.astype(float)
X = (X - X.mean(axis=0)) / X.std(axis=0)

y, centers = kmeans(X[:, 2:], 5)

np.random.seed(42)
index = np.random.permutation(len(X))
split = int(0.8 * len(X))
X_train, X_test = X[index[:split]], X[index[split:]]
y_train, y_test = y[index[:split]], y[index[split:]]

y_pred = knn_predict(X_train, y_train, X_test, k=5)

print("Accuracy:", np.mean(y_test == y_pred))
print("Confusion Matrix:")
print(pd.crosstab(y_test, y_pred, rownames=["Actual"], colnames=["Predicted"]))
```

Note: Mall_Customers.csv has no segment column, so segments are first made with K-Means (5 clusters) and KNN then classifies them. KNN and K-Means are written with numpy.

Slip 7

Q1 [10 Marks]

Bar Chart of students enrolled: courses = [BCA, BBA, B.Com, B.Sc, BA], students = [60, 45, 75, 50, 35]. Use different bar colors, add a title and label both axes.

```
import matplotlib.pyplot as plt

courses = ["BCA", "BBA", "B.Com", "B.Sc", "BA"]
students = [60, 45, 75, 50, 35]
colors = ["blue", "orange", "green", "red", "purple"]

plt.bar(courses, students, color=colors)
plt.title("Number of Students Enrolled in Each Course")
plt.xlabel("Courses")
plt.ylabel("Number of Students")
plt.show()
```

Q2 [20 Marks]

Implement a KNN classifier on the Student dataset. Preprocess the data and classify students into performance categories. Evaluate with Accuracy and Confusion Matrix.

```
import numpy as np
import pandas as pd

def knn_predict(X_train, y_train, X_test, k=5):
    predictions = []
    for row in X_test:
        distances = np.sqrt(((X_train - row) ** 2).sum(axis=1))
        nearest = y_train[np.argsort(distances)[:k]]
        values, counts = np.unique(nearest, return_counts=True)
        predictions.append(values[counts.argmax()])
    return np.array(predictions)


df = pd.read_csv("student_dataset_v2.csv")
print(df.head())
print(df.isnull().sum())
df = df.dropna()

for col in df.select_dtypes(exclude="number").columns:
    df[col] = df[col].astype("category").cat.codes

X = df.iloc[:, :-1].values.astype(float)
y = df.iloc[:, -1].values.astype(int)

np.random.seed(42)
index = np.random.permutation(len(X))
split = int(0.8 * len(X))
X_train, X_test = X[index[:split]], X[index[split:]]
y_train, y_test = y[index[:split]], y[index[split:]]

mean = X_train.mean(axis=0)
std = X_train.std(axis=0) + 1e-9
X_train = (X_train - mean) / std
X_test = (X_test - mean) / std

y_pred = knn_predict(X_train, y_train, X_test, k=5)

print("Accuracy:", np.mean(y_test == y_pred))
print("Confusion Matrix:")
print(pd.crosstab(y_test, y_pred, rownames=["Actual"], colnames=["Predicted"]))
```

Note: The last column is taken as the performance category (target). If your file has an ID column, drop it first. KNN (k = 5) is written with numpy.

Q2 OR [20 Marks]

Market basket analysis on the Groceries dataset using Apriori: read and explore the data, handle missing values, convert to a numeric format, generate frequent itemsets and association rules for minimum support 0.3 and 0.4.

```
import pandas as pd
from itertools import combinations

def apriori(transactions, min_support):
```

```
    n = len(transactions)
    items = sorted(set(i for t in transactions for i in t))
    frequent = {}
    current = [(i,) for i in items]
    while current:
        found = {}
        for itemset in current:
            count = sum(1 for t in transactions if set(itemset).issubset(t))
            if count / n >= min_support:
                found[itemset] = count / n
        frequent.update(found)
        keys = list(found)
        current = set()
        for a in range(len(keys)):
            for b in range(a + 1, len(keys)):
                union = tuple(sorted(set(keys[a]) | set(keys[b])))
                if len(union) == len(keys[a]) + 1:
                    current.add(union)
        current = list(current)
    return frequent


def association_rules(frequent, min_confidence):
    rules = []
    for itemset, support in frequent.items():
        if len(itemset) < 2:
            continue
        for r in range(1, len(itemset)):
            for antecedent in combinations(itemset, r):
                consequent = tuple(i for i in itemset if i not in antecedent)
                confidence = support / frequent[antecedent]
                lift = confidence / frequent[consequent]
                if confidence >= min_confidence:
                    rules.append([antecedent, consequent, support, confidence, lift])
    return pd.DataFrame(rules, columns=["antecedent", "consequent", "support", "confidence", "lift"])


df = pd.DataFrame({"Items": [
    "Basmati Rice, Toor Dal, Turmeric Powder, Cooking Oil",
    "Atta, Sugar, Tea, Milk",
    "Rice, Moong Dal, Salt, Cooking Oil",
    "Wheat Flour, Chickpeas, Turmeric Powder, Cumin Seeds",
    "Poha, Peanuts, Onions, Cooking Oil",
    "Basmati Rice, Paneer, Garam Masala, Tomatoes",
    "Atta, Potatoes, Onions, Cooking Oil"]})
print(df)
print(df.isnull().sum())
df = df.dropna()

transactions = [row.split(", ") for row in df["Items"]]

for support in [0.3, 0.4]:
    print("Minimum support =", support)
    frequent = apriori(transactions, support)
    print(pd.DataFrame(list(frequent.items()), columns=["itemsets", "support"]))
    print(association_rules(frequent, 0.5))
```

Note: With these 7 transactions only Cooking Oil is frequent at 0.3 and 0.4, so the rules table will be empty. That is the correct result. Apriori is written with itertools and pandas.

Slip 11

Q1 [10 Marks]

Create a bar chart showing the distribution of employees among HR, Finance, IT, Sales and Marketing with suitable labels and a title.

```
import matplotlib.pyplot as plt

departments = ["HR", "Finance", "IT", "Sales", "Marketing"]
employees = [15, 25, 60, 45, 30]

plt.bar(departments, employees, color="skyblue")
plt.title("Number of Employees in Each Department")
plt.xlabel("Department")
plt.ylabel("Number of Employees")
plt.show()
```

Q2 [20 Marks]

Apply Logistic Regression to classify smartphones into price categories using the smartphone dataset. Preprocess and assess with Accuracy, F1-Score and Confusion Matrix.

```
import numpy as np
import pandas as pd

df = pd.read_csv("train.csv")
print(df.info())
df = df.dropna()

X = df.drop("price_range", axis=1).values.astype(float)
y = df["price_range"].values.astype(int)

np.random.seed(42)
index = np.random.permutation(len(X))
split = int(0.8 * len(X))
X_train, X_test = X[index[:split]], X[index[split:]]
y_train, y_test = y[index[:split]], y[index[split:]]

mean = X_train.mean(axis=0)
std = X_train.std(axis=0) + 1e-9
X_train = (X_train - mean) / std
X_test = (X_test - mean) / std

classes = np.unique(y_train)
Y = (y_train[:, None] == classes).astype(float)
W = np.zeros((X_train.shape[1], len(classes)))
b = np.zeros(len(classes))

for i in range(1000):
    scores = X_train @ W + b
    e = np.exp(scores - scores.max(axis=1, keepdims=True))
    P = e / e.sum(axis=1, keepdims=True)
    W = W - 0.1 * (X_train.T @ (P - Y)) / len(y_train)
    b = b - 0.1 * (P - Y).mean(axis=0)

y_pred = classes[np.argmax(X_test @ W + b, axis=1)]

classes = np.unique(y_test)
precision = []
recall = []
for c in classes:
    tp = np.sum((y_test == c) & (y_pred == c))
    fp = np.sum((y_test != c) & (y_pred == c))
    fn = np.sum((y_test == c) & (y_pred != c))
    precision.append(tp / (tp + fp) if tp + fp > 0 else 0)
    recall.append(tp / (tp + fn) if tp + fn > 0 else 0)

precision = np.mean(precision)
recall = np.mean(recall)
f1 = 2 * precision * recall / (precision + recall)

print("Accuracy :", np.mean(y_test == y_pred))
print("Precision:", precision)
print("Recall   :", recall)
print("F1-Score :", f1)
print("Confusion Matrix:")
```

```
print(pd.crosstab(y_test, y_pred, rownames=["Actual"], colnames=["Predicted"]))
```

Note: Uses the Mobile Price Classification train.csv (target: price_range; features include ram, int_memory, battery_power and pc).

Q2 OR [20 Marks]

Apply K-Means Clustering on the E-commerce Customer dataset. Read, inspect and preprocess (missing values, numerical features), segment the customers and display the clusters.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

def kmeans(X, k, iterations=100):
    np.random.seed(42)
    centers = X[np.random.choice(len(X), k, replace=False)]
    for i in range(iterations):
        distances = np.sqrt(((X[:, None, :] - centers[None, :, :]) ** 2).sum(axis=2))
        labels = distances.argmin(axis=1)
        new_centers = np.array([X[labels == j].mean(axis=0) if np.any(labels == j)
                                else centers[j] for j in range(k)])
        if np.allclose(new_centers, centers):
            break
        centers = new_centers
    return labels, centers


df = pd.read_csv("Ecommerce Customers.csv")
print(df.info())
print(df.isnull().sum())
df = df.dropna()

features = ["Avg. Session Length", "Time on App", "Time on Website",
            "Length of Membership", "Yearly Amount Spent"]
X = df[features].values.astype(float)
X = (X - X.mean(axis=0)) / X.std(axis=0)

labels, centers = kmeans(X, 3)
df["Cluster"] = labels

print(df["Cluster"].value_counts())
print(df.groupby("Cluster")[features].mean())

plt.scatter(X[:, 3], X[:, 4], c=labels)
plt.scatter(centers[:, 3], centers[:, 4], color="red", marker="X", s=200, label="Centroids")
plt.xlabel("Length of Membership")
plt.ylabel("Yearly Amount Spent")
plt.title("Customer Clusters")
plt.legend()
plt.show()
```

Slip 14

Q1 [10 Marks]

Create a DataFrame of products with Price, Rating and Number of Sales. Generate a Dendrogram to group similar products.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

df = pd.DataFrame({
    "Product": ["Laptop", "Phone", "Tablet", "Headphones", "Smartwatch", "Camera", "Speaker", "Mouse"],
    "Price": [55000, 30000, 25000, 3000, 8000, 40000, 5000, 800],
    "Rating": [4.5, 4.4, 4.2, 4.0, 4.1, 4.6, 3.9, 4.0],
    "Number_of_Sales": [1200, 5000, 1800, 6000, 3500, 700, 4200, 7000]
})
print(df)

X = df[["Price", "Rating", "Number_of_Sales"]].values.astype(float)
X = (X - X.mean(axis=0)) / X.std(axis=0)
n = len(X)
dist = np.sqrt(((X[:, None, :] - X[None, :, :]) ** 2).sum(axis=2))

nodes = {i: i for i in range(n)}
members = {i: [i] for i in range(n)}
next_id = n
while len(nodes) > 1:
    best = None
    ids = list(nodes)
    for a in range(len(ids)):
        for b in range(a + 1, len(ids)):
            d = dist[np.ix_(members[ids[a]], members[ids[b]])].mean()
            if best is None or d < best[0]:
                best = (d, ids[a], ids[b])
    d, a, b = best
    nodes[next_id] = (nodes.pop(a), nodes.pop(b), d)
    members[next_id] = members.pop(a) + members.pop(b)
    next_id += 1
root = nodes[next_id - 1]


def leaves(node):
    if isinstance(node, int):
        return [node]
    return leaves(node[0]) + leaves(node[1])


order = leaves(root)
position = {leaf: i for i, leaf in enumerate(order)}


def draw(node):
    if isinstance(node, int):
        return position[node], 0
    xl, hl = draw(node[0])
    xr, hr = draw(node[1])
    plt.plot([xl, xl, xr, xr], [hl, node[2], node[2], hr], color="blue")
    return (xl + xr) / 2, node[2]


draw(root)
plt.xticks(range(n), df["Product"].values[order], rotation=90)
plt.title("Dendrogram of Products")
plt.xlabel("Products")
plt.ylabel("Distance")
plt.tight_layout()
plt.show()
```

Q2 [20 Marks]

Using the Car Price Prediction dataset, apply Polynomial Regression to predict Selling Price from Vehicle Age, Present Price and Kilometers Driven. Evaluate with R2 score and visualize the actual data points with the fitted polynomial curve.

```
import numpy as np
```

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("car data.csv")
df = df.dropna()
df["Vehicle_Age"] = 2026 - df["Year"]

X = df[["Vehicle_Age", "Present_Price", "Kms_Driven"]].values.astype(float)
y = df["Selling_Price"].values.astype(float)

np.random.seed(42)
index = np.random.permutation(len(X))
split = int(0.8 * len(X))
X_train, X_test = X[index[:split]], X[index[split:]]
y_train, y_test = y[index[:split]], y[index[split:]]

present_price = X_test[:, 1].copy()
mean = X_train.mean(axis=0)
std = X_train.std(axis=0) + 1e-9
X_train = (X_train - mean) / std
X_test = (X_test - mean) / std


def poly_features(X):
    columns = [np.ones(len(X))]
    for i in range(X.shape[1]):
        columns.append(X[:, i])
    for i in range(X.shape[1]):
        for j in range(i, X.shape[1]):
            columns.append(X[:, i] * X[:, j])
    return np.column_stack(columns)


coef = np.linalg.lstsq(poly_features(X_train), y_train, rcond=None)[0]
y_pred = poly_features(X_test) @ coef

ss_res = np.sum((y_test - y_pred) ** 2)
ss_tot = np.sum((y_test - y_test.mean()) ** 2)
print("R2 Score:", 1 - ss_res / ss_tot)

order = np.argsort(present_price)
plt.scatter(present_price, y_test, color="blue", label="Actual")
plt.plot(present_price[order], y_pred[order], color="red", label="Polynomial fit")
plt.xlabel("Present Price")
plt.ylabel("Selling Price")
plt.title("Polynomial Regression")
plt.legend()
plt.show()
```

Note: Vehicle_Age = 2026 minus Year. Polynomial (degree 2) features are built with numpy and solved with np.linalg.lstsq.

Q2 OR [20 Marks]

Create two DataFrames of mobile phones (DataFrame 1: Brand, Model, Price, RAM; DataFrame 2: Model, Storage, Battery_Capacity, Operating_System). Perform (a) data integration on Model, (b) Label Encoding on Brand, (c) One-Hot Encoding on Operating_System. Display the original, merged and transformed DataFrames.

```
import pandas as pd

df1 = pd.DataFrame({
    "Brand": ["Samsung", "Apple", "OnePlus", "Xiaomi"],
    "Model": ["Galaxy S23", "iPhone 14", "OnePlus 11", "Redmi Note 12"],
    "Price": [74999, 79900, 56999, 17999],
    "RAM": [8, 6, 12, 6]
})
df2 = pd.DataFrame({
    "Model": ["Galaxy S23", "iPhone 14", "OnePlus 11", "Redmi Note 12"],
    "Storage": [256, 128, 256, 128],
    "Battery_Capacity": [3900, 3279, 5000, 5000],
    "Operating_System": ["Android", "iOS", "Android", "Android"]
})
print(df1)
print(df2)

merged = pd.merge(df1, df2, on="Model")
print("Merged DataFrame:")
print(merged)
```

```
merged["Brand"] = merged["Brand"].astype("category").cat.codes
transformed = pd.get_dummies(merged, columns=["Operating_System"])
print("Transformed DataFrame:")
print(transformed)
```

Slip 15

Q1 [10 Marks]

Create an age_records DataFrame with Person_ID and Age. Perform Data Discretization on Age using Equal Frequency Binning. Display the original and discretized data.

```
import pandas as pd

age_records = pd.DataFrame({
    "Person_ID": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12],
    "Age": [18, 22, 25, 27, 31, 35, 38, 42, 47, 53, 61, 70]
})

age_records["Age_Group"] = pd.qcut(age_records["Age"], q=3, labels=["Low", "Medium", "High"])

print(age_records)
print(age_records["Age_Group"].value_counts())
```

Q2 [20 Marks]

Create a DataFrame of cities with City, Area (sq. km), Average Temperature, Rainfall and Population Density. Generate a Treemap of the population density of each city and a 3D Scatter Plot using Area, Average Temperature and Rainfall, with labels, colors and titles.

```
import pandas as pd
import matplotlib.pyplot as plt

def treemap(sizes, labels, x, y, w, h, colors, vertical=True):
    if len(sizes) == 0:
        return
    share = sizes[0] / sum(sizes)
    if len(sizes) == 1:
        share = 1
    if vertical:
        rw, rh = w * share, h
    else:
        rw, rh = w, h * share
    plt.gca().add_patch(plt.Rectangle((x, y), rw, rh, facecolor=colors[0], edgecolor="white"))
    plt.text(x + rw / 2, y + rh / 2, labels[0], ha="center", va="center", color="white")
    if vertical:
        treemap(sizes[1:], labels[1:], x + rw, y, w - rw, h, colors[1:], False)
    else:
        treemap(sizes[1:], labels[1:], x, y + rh, w, h - rh, colors[1:], True)


df = pd.DataFrame({
    "City": ["Mumbai", "Delhi", "Bengaluru", "Chennai", "Pune"],
    "Area": [603, 1484, 741, 426, 331],
    "Average_Temperature": [27, 25, 24, 29, 25],
    "Rainfall": [2400, 800, 970, 1400, 722],
    "Population_Density": [20700, 11300, 11800, 26900, 9400]
})
print(df)

df = df.sort_values("Population_Density", ascending=False)
labels = [c + "\n" + str(d) for c, d in zip(df["City"], df["Population_Density"])]

plt.figure()
treemap(list(df["Population_Density"]), labels, 0, 0, 100, 100,
        ["red", "green", "blue", "orange", "purple"])
plt.xlim(0, 100)
plt.ylim(0, 100)
plt.axis("off")
plt.title("Treemap of Population Density")
plt.show()

fig = plt.figure()
ax = fig.add_subplot(111, projection="3d")
ax.scatter(df["Area"], df["Average_Temperature"], df["Rainfall"], c="red", s=60)
ax.set_xlabel("Area (sq. km)")
ax.set_ylabel("Average Temperature")
ax.set_zlabel("Rainfall (mm)")
ax.set_title("3D Scatter Plot")
plt.show()
```

Q2 OR [20 Marks]

Market basket analysis on the Groceries dataset using Apriori: explore, handle missing values, convert to a numeric format, generate frequent itemsets and rules for minimum support 0.3 and 0.4.

```
import pandas as pd
from itertools import combinations

def apriori(transactions, min_support):
    n = len(transactions)
    items = sorted(set(i for t in transactions for i in t))
    frequent = {}
    current = [(i,) for i in items]
    while current:
        found = {}
        for itemset in current:
            count = sum(1 for t in transactions if set(itemset).issubset(t))
            if count / n >= min_support:
                found[itemset] = count / n
        frequent.update(found)
        keys = list(found)
        current = set()
        for a in range(len(keys)):
            for b in range(a + 1, len(keys)):
                union = tuple(sorted(set(keys[a]) | set(keys[b])))
                if len(union) == len(keys[a]) + 1:
                    current.add(union)
        current = list(current)
    return frequent


def association_rules(frequent, min_confidence):
    rules = []
    for itemset, support in frequent.items():
        if len(itemset) < 2:
            continue
        for r in range(1, len(itemset)):
            for antecedent in combinations(itemset, r):
                consequent = tuple(i for i in itemset if i not in antecedent)
                confidence = support / frequent[antecedent]
                lift = confidence / frequent[consequent]
                if confidence >= min_confidence:
                    rules.append([antecedent, consequent, support, confidence, lift])
    return pd.DataFrame(rules, columns=["antecedent", "consequent", "support", "confidence", "lift"])


df = pd.DataFrame({"Items": [
    "Basmati Rice, Toor Dal, Turmeric Powder, Cooking Oil",
    "Atta, Sugar, Tea, Milk",
    "Rice, Moong Dal, Salt, Cooking Oil",
    "Wheat Flour, Chickpeas, Turmeric Powder, Cumin Seeds",
    "Poha, Peanuts, Onions, Cooking Oil",
    "Basmati Rice, Paneer, Garam Masala, Tomatoes",
    "Atta, Potatoes, Onions, Cooking Oil"]})
print(df)
print(df.isnull().sum())
df = df.dropna()

transactions = [row.split(", ") for row in df["Items"]]

for support in [0.3, 0.4]:
    print("Minimum support =", support)
    frequent = apriori(transactions, support)
    print(pd.DataFrame(list(frequent.items()), columns=["itemsets", "support"]))
    print(association_rules(frequent, 0.5))
```

Note: With these 7 transactions only Cooking Oil is frequent at 0.3 and 0.4, so the rules table will be empty. That is the correct result. Apriori is written with itertools and pandas.

Slip 16

Q1 [10 Marks]

Create a DataFrame of student performance data (Study Hours, Attendance, Assignment Score, Exam Score). Calculate the correlation matrix and generate a Heat Map with the correlation values, labels and a title.

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.DataFrame({
    "Study_Hours": [2, 4, 6, 8, 3, 5, 7, 9],
    "Attendance": [60, 70, 80, 90, 65, 75, 85, 95],
    "Assignment_Score": [55, 65, 75, 88, 60, 70, 82, 92],
    "Exam_Score": [50, 62, 78, 90, 58, 68, 84, 95]
})

corr = df.corr()
print(corr)

plt.imshow(corr, cmap="coolwarm", vmin=-1, vmax=1)
plt.colorbar()
plt.xticks(range(len(corr)), corr.columns, rotation=45)
plt.yticks(range(len(corr)), corr.columns)
for i in range(len(corr)):
    for j in range(len(corr)):
        plt.text(j, i, round(corr.iloc[i, j], 2), ha="center", va="center")
plt.title("Correlation Heat Map")
plt.tight_layout()
plt.show()
```

Q2 [20 Marks]

Create a DataFrame with Temperature, Humidity, WindSpeed, Pressure and Rainfall. Apply (a) Min-Max Scaling, (b) Standardization and (c) Normalization and display the transformed data.

```
import pandas as pd

df = pd.DataFrame({
    "Temperature": [30.5, 28.0, 35.2, 22.1, 25.6, 33.0],
    "Humidity": [65, 70, 50, 85, 78, 55],
    "WindSpeed": [12, 15, 8, 20, 18, 9],
    "Pressure": [1010, 1008, 1005, 1015, 1012, 1006],
    "Rainfall": [5, 12, 0, 30, 20, 1]
})
print("Original data:")
print(df)

minmax = (df - df.min()) / (df.max() - df.min())
print("Min-Max Scaling:")
print(minmax)

standard = (df - df.mean()) / df.std(ddof=0)
print("Standardization:")
print(standard)

normal = df.div(((df ** 2).sum(axis=1)) ** 0.5, axis=0)
print("Normalization:")
print(normal)
```

Q2 OR [20 Marks]

Implement Apriori on the student activity dataset (9 transactions) to find frequent itemsets and association rules. Repeat with minimum support 0.2 and 0.3.

```
import pandas as pd
from itertools import combinations

def apriori(transactions, min_support):
    n = len(transactions)
    items = sorted(set(i for t in transactions for i in t))
    frequent = {}
    current = [(i,) for i in items]
    while current:
        found = {}
        for itemset in current:
```

```
            count = sum(1 for t in transactions if set(itemset).issubset(t))
            if count / n >= min_support:
                found[itemset] = count / n
        frequent.update(found)
        keys = list(found)
        current = set()
        for a in range(len(keys)):
            for b in range(a + 1, len(keys)):
                union = tuple(sorted(set(keys[a]) | set(keys[b])))
                if len(union) == len(keys[a]) + 1:
                    current.add(union)
        current = list(current)
    return frequent


def association_rules(frequent, min_confidence):
    rules = []
    for itemset, support in frequent.items():
        if len(itemset) < 2:
            continue
        for r in range(1, len(itemset)):
            for antecedent in combinations(itemset, r):
                consequent = tuple(i for i in itemset if i not in antecedent)
                confidence = support / frequent[antecedent]
                lift = confidence / frequent[consequent]
                if confidence >= min_confidence:
                    rules.append([antecedent, consequent, support, confidence, lift])
    return pd.DataFrame(rules, columns=["antecedent", "consequent", "support", "confidence", "lift"])


transactions = [
    ["Library", "Seminar"],
    ["Workshop", "Project"],
    ["Library", "Assignment"],
    ["Seminar", "Workshop"],
    ["Assignment", "Seminar"],
    ["Workshop", "Assignment"],
    ["Project", "Library"],
    ["Library", "Seminar", "Project"],
    ["Library", "Seminar", "Workshop"]
]

for support in [0.2, 0.3]:
    print("Minimum support =", support)
    frequent = apriori(transactions, support)
    print(pd.DataFrame(list(frequent.items()), columns=["itemsets", "support"]))
    print(association_rules(frequent, 0.5))
```

Slip 17

Q1 [10 Marks]

Create two DataFrames student_details and academic_records and integrate them using Student_ID. Display the individual and integrated DataFrames.

```
import pandas as pd

student_details = pd.DataFrame({
    "Student_ID": [101, 102, 103, 104, 105],
    "Name": ["Aarav", "Diya", "Kabir", "Isha", "Rohan"],
    "City": ["Nashik", "Pune", "Mumbai", "Nashik", "Nagpur"]
})
academic_records = pd.DataFrame({
    "Student_ID": [101, 102, 103, 104, 105],
    "Course": ["B.Sc CS", "B.Sc CS", "BCA", "B.Sc CS", "BBA"],
    "Marks": [85, 78, 65, 91, 72]
})
print("student_details:")
print(student_details)
print("academic_records:")
print(academic_records)

integrated = pd.merge(student_details, academic_records, on="Student_ID")
print("Integrated DataFrame:")
print(integrated)
```

Q2 [20 Marks]

Create a dataset of cities with City, Population, GDP and Literacy Rate. Generate (a) a Treemap of city-wise population and (b) a 3D Scatter Plot using Population, GDP and Literacy Rate.

```
import pandas as pd
import matplotlib.pyplot as plt

def treemap(sizes, labels, x, y, w, h, colors, vertical=True):
    if len(sizes) == 0:
        return
    share = sizes[0] / sum(sizes)
    if len(sizes) == 1:
        share = 1
    if vertical:
        rw, rh = w * share, h
    else:
        rw, rh = w, h * share
    plt.gca().add_patch(plt.Rectangle((x, y), rw, rh, facecolor=colors[0], edgecolor="white"))
    plt.text(x + rw / 2, y + rh / 2, labels[0], ha="center", va="center", color="white")
    if vertical:
        treemap(sizes[1:], labels[1:], x + rw, y, w - rw, h, colors[1:], False)
    else:
        treemap(sizes[1:], labels[1:], x, y + rh, w, h - rh, colors[1:], True)


df = pd.DataFrame({
    "City": ["Mumbai", "Delhi", "Bengaluru", "Chennai", "Pune"],
    "Population": [12442373, 11034555, 8443675, 4646732, 3124458],
    "GDP": [310, 293, 110, 78, 69],
    "Literacy_Rate": [89.7, 86.2, 88.7, 90.2, 91.6]
})
print(df)

df = df.sort_values("Population", ascending=False)

plt.figure()
treemap(list(df["Population"]), list(df["City"]), 0, 0, 100, 100,
        ["red", "green", "blue", "orange", "purple"])
plt.xlim(0, 100)
plt.ylim(0, 100)
plt.axis("off")
plt.title("Treemap of City-wise Population")
plt.show()

fig = plt.figure()
ax = fig.add_subplot(111, projection="3d")
ax.scatter(df["Population"], df["GDP"], df["Literacy_Rate"], c="blue", s=60)
ax.set_xlabel("Population")
```

```
ax.set_ylabel("GDP")
ax.set_zlabel("Literacy Rate")
ax.set_title("3D Scatter Plot")
plt.show()
```

Q2 OR [20 Marks]

Develop a Random Forest Classifier on the Student Grading dataset to classify students into performance categories. Preprocess, train, test and evaluate with Confusion Matrix, Accuracy, Precision, Recall and F1-Score.

```
import numpy as np
import pandas as pd

def gini(y):
    counts = np.bincount(y)
    p = counts[counts > 0] / len(y)
    return 1 - np.sum(p ** 2)


def build_tree(X, y, depth, n_features):
    if depth == 8 or len(np.unique(y)) == 1 or len(y) < 5:
        return np.bincount(y).argmax()
    best = None
    best_score = 1
    for f in np.random.choice(X.shape[1], n_features, replace=False):
        for t in np.unique(np.percentile(X[:, f], [10, 25, 50, 75, 90])):
            left = X[:, f] <= t
            if left.sum() == 0 or left.sum() == len(y):
                continue
            score = (left.sum() * gini(y[left]) + (~left).sum() * gini(y[~left])) / len(y)
            if score < best_score:
                best_score = score
                best = (f, t)
    if best is None:
        return np.bincount(y).argmax()
    f, t = best
    left = X[:, f] <= t
    return (f, t, build_tree(X[left], y[left], depth + 1, n_features),
            build_tree(X[~left], y[~left], depth + 1, n_features))


def predict_tree(tree, row):
    while isinstance(tree, tuple):
        f, t, left, right = tree
        tree = left if row[f] <= t else right
    return tree


df = pd.read_csv("Students_Grading_Dataset.csv")
print(df.isnull().sum())

df = df.drop(["Student_ID", "First_Name", "Last_Name", "Email", "Total_Score"], axis=1)
df = df.dropna()

for col in df.select_dtypes(exclude="number").columns:
    df[col] = df[col].astype("category").cat.codes

X = df.drop("Grade", axis=1).values.astype(float)
y = df["Grade"].values.astype(int)

np.random.seed(42)
index = np.random.permutation(len(X))
split = int(0.8 * len(X))
X_train, X_test = X[index[:split]], X[index[split:]]
y_train, y_test = y[index[:split]], y[index[split:]]

np.random.seed(42)
trees = []
for i in range(10):
    sample = np.random.randint(0, len(X_train), len(X_train))
    trees.append(build_tree(X_train[sample], y_train[sample], 0, int(np.sqrt(X_train.shape[1])) + 1))

votes = np.array([[predict_tree(tree, row) for row in X_test] for tree in trees])
y_pred = np.array([np.bincount(votes[:, j]).argmax() for j in range(votes.shape[1])])

classes = np.unique(y_test)
precision = []
recall = []
```

```
for c in classes:
    tp = np.sum((y_test == c) & (y_pred == c))
    fp = np.sum((y_test != c) & (y_pred == c))
    fn = np.sum((y_test == c) & (y_pred != c))
    precision.append(tp / (tp + fp) if tp + fp > 0 else 0)
    recall.append(tp / (tp + fn) if tp + fn > 0 else 0)

precision = np.mean(precision)
recall = np.mean(recall)
f1 = 2 * precision * recall / (precision + recall)

print("Accuracy :", np.mean(y_test == y_pred))
print("Precision:", precision)
print("Recall   :", recall)
print("F1-Score :", f1)
print("Confusion Matrix:")
print(pd.crosstab(y_test, y_pred, rownames=["Actual"], colnames=["Predicted"]))
```

Note: Grade is the target. Total_Score is dropped because the grade is calculated from it. The Random Forest is 10 small decision trees built with numpy (gini index, bootstrap samples, majority vote). Precision, Recall and F1 are macro averages.

Slip 18

Q1 [10 Marks]

Create an employee_data DataFrame (Emp_ID, Age, Dept, Years_Experience, Performance_Rating). Perform data cleaning: handle missing values, remove duplicates and detect outliers using the IQR method.

```
import pandas as pd

employee_data = pd.DataFrame({
    "Emp_ID": [101, 102, 103, 104, 105, 106, 106, 107, 108],
    "Age": [25, 30, None, 45, 38, 29, 29, 75, 33],
    "Dept": ["HR", "IT", "Finance", None, "IT", "Sales", "Sales", "IT", "HR"],
    "Years_Experience": [2, 6, 8, 20, None, 4, 4, 40, 7],
    "Performance_Rating": [3.5, 4.0, 3.8, 4.2, 3.9, 4.1, 4.1, 2.0, 3.7]
})
print(employee_data)
print(employee_data.isnull().sum())

employee_data["Age"] = employee_data["Age"].fillna(employee_data["Age"].mean())
employee_data["Years_Experience"] = employee_data["Years_Experience"].fillna(employee_data["Years_Experience"].mean())
employee_data["Dept"] = employee_data["Dept"].fillna(employee_data["Dept"].mode()[0])

employee_data = employee_data.drop_duplicates()
print("After cleaning:")
print(employee_data)

for col in ["Age", "Years_Experience"]:
    Q1 = employee_data[col].quantile(0.25)
    Q3 = employee_data[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    outliers = employee_data[(employee_data[col] < lower) | (employee_data[col] > upper)]
    print("Outliers in", col)
    print(outliers)
```

Q2 [20 Marks]

World Cities dataset (city, country, latitude, longitude, population): (a) Geospatial Visualization on a world map with marker size showing population, (b) Treemap of the population distribution.

```
import pandas as pd
import matplotlib.pyplot as plt

def treemap(sizes, labels, x, y, w, h, colors, vertical=True):
    if len(sizes) == 0:
        return
    share = sizes[0] / sum(sizes)
    if len(sizes) == 1:
        share = 1
    if vertical:
        rw, rh = w * share, h
    else:
        rw, rh = w, h * share
    plt.gca().add_patch(plt.Rectangle((x, y), rw, rh, facecolor=colors[0], edgecolor="white"))
    plt.text(x + rw / 2, y + rh / 2, labels[0], ha="center", va="center", color="white")
    if vertical:
        treemap(sizes[1:], labels[1:], x + rw, y, w - rw, h, colors[1:], False)
    else:
        treemap(sizes[1:], labels[1:], x, y + rh, w, h - rh, colors[1:], True)


df = pd.read_csv("worldcities.csv")
print(df.columns)
df = df.dropna()

plt.scatter(df["longitude"], df["latitude"], s=df["population"] / 100000, alpha=0.5, color="red")
plt.title("World Cities (marker size = population)")
plt.xlabel("Longitude")
plt.ylabel("Latitude")
plt.show()

top = df.sort_values("population", ascending=False).head(10)
colors = ["red", "green", "blue", "orange", "purple", "brown", "pink", "gray", "olive", "cyan"]

plt.figure()
```

```
treemap(list(top["population"]), list(top["city"]), 0, 0, 100, 100, colors)
plt.xlim(0, 100)
plt.ylim(0, 100)
plt.axis("off")
plt.title("Treemap of Population (Top 10 Cities)")
plt.show()
```

Note: The map is drawn as a longitude-latitude scatter plot. If your file uses lat / lng, change the column names.

Q2 OR [20 Marks]

Use the Crop Yield dataset to apply Simple Linear Regression predicting crop yield from rainfall. Handle missing values, split into train / test, evaluate with MAE and R2 and plot a scatter plot with the regression line.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("crop_yield_data.csv")
print(df.columns)
print(df.isnull().sum())
df = df.fillna(df.mean(numeric_only=True))

X = df["Rainfall"].values
y = df["Yield"].values

np.random.seed(42)
index = np.random.permutation(len(X))
split = int(0.8 * len(X))
X_train, X_test = X[index[:split]], X[index[split:]]
y_train, y_test = y[index[:split]], y[index[split:]]

slope, intercept = np.polyfit(X_train, y_train, 1)
y_pred = slope * X_test + intercept

ss_res = np.sum((y_test - y_pred) ** 2)
ss_tot = np.sum((y_test - y_test.mean()) ** 2)
print("MAE     :", np.mean(np.abs(y_test - y_pred)))
print("R2 Score:", 1 - ss_res / ss_tot)

line_x = np.sort(X_test)
plt.scatter(X_test, y_test, color="green", label="Actual")
plt.plot(line_x, slope * line_x + intercept, color="red", label="Regression Line")
plt.xlabel("Rainfall")
plt.ylabel("Crop Yield")
plt.title("Rainfall vs Crop Yield")
plt.legend()
plt.show()
```

Note: Change the column names Rainfall / Yield if your file uses different names.

Slip 19

Q1 [10 Marks]

Create a DataFrame of vehicles (Fuel_Efficiency, Engine_Power, Vehicle_Weight, Engine_Size, Acceleration) with 5 records including an outlier. Create Box Plots for Fuel Efficiency and Engine Power and identify the outliers.

```
import pandas as pd
import matplotlib.pyplot as plt

vehicles = pd.DataFrame({
    "Fuel_Efficiency": [18.5, 20.0, 17.2, 19.1, 45.0],
    "Engine_Power": [110, 120, 105, 115, 450],
    "Vehicle_Weight": [1200, 1300, 1150, 1250, 2600],
    "Engine_Size": [1.2, 1.5, 1.2, 1.4, 5.0],
    "Acceleration": [12.5, 11.8, 13.0, 12.2, 4.5]
})
print(vehicles)

vehicles[["Fuel_Efficiency"]].boxplot()
plt.title("Box Plot of Fuel Efficiency")
plt.show()

vehicles[["Engine_Power"]].boxplot()
plt.title("Box Plot of Engine Power")
plt.show()

for col in ["Fuel_Efficiency", "Engine_Power"]:
    Q1 = vehicles[col].quantile(0.25)
    Q3 = vehicles[col].quantile(0.75)
    IQR = Q3 - Q1
    outliers = vehicles[(vehicles[col] < Q1 - 1.5 * IQR) | (vehicles[col] > Q3 + 1.5 * IQR)]
    print("Outliers in", col)
    print(outliers[col])
```

Q2 [20 Marks]

Create a DataFrame with Temperature, Humidity, Wind Speed, Pressure and Rainfall. Apply (a) Min-Max Scaling, (b) Standardization and (c) Normalization and display the transformed data.

```
import pandas as pd

df = pd.DataFrame({
    "Temperature": [30.5, 28.0, 35.2, 22.1, 25.6, 33.0],
    "Humidity": [65, 70, 50, 85, 78, 55],
    "WindSpeed": [12, 15, 8, 20, 18, 9],
    "Pressure": [1010, 1008, 1005, 1015, 1012, 1006],
    "Rainfall": [5, 12, 0, 30, 20, 1]
})
print("Original data:")
print(df)

minmax = (df - df.min()) / (df.max() - df.min())
print("Min-Max Scaling:")
print(minmax)

standard = (df - df.mean()) / df.std(ddof=0)
print("Standardization:")
print(standard)

normal = df.div(((df ** 2).sum(axis=1)) ** 0.5, axis=0)
print("Normalization:")
print(normal)
```

Q2 OR [20 Marks]

Implement Random Forest Classification on the Heart Disease dataset to predict presence or absence of heart disease. Preprocess, train, test and evaluate with Accuracy, Precision, Recall, F1-Score and Confusion Matrix.

```
import numpy as np
import pandas as pd

def gini(y):
    counts = np.bincount(y)
    p = counts[counts > 0] / len(y)
    return 1 - np.sum(p ** 2)
```

```
def build_tree(X, y, depth, n_features):
    if depth == 8 or len(np.unique(y)) == 1 or len(y) < 5:
        return np.bincount(y).argmax()
    best = None
    best_score = 1
    for f in np.random.choice(X.shape[1], n_features, replace=False):
        for t in np.unique(np.percentile(X[:, f], [10, 25, 50, 75, 90])):
            left = X[:, f] <= t
            if left.sum() == 0 or left.sum() == len(y):
                continue
            score = (left.sum() * gini(y[left]) + (~left).sum() * gini(y[~left])) / len(y)
            if score < best_score:
                best_score = score
                best = (f, t)
    if best is None:
        return np.bincount(y).argmax()
    f, t = best
    left = X[:, f] <= t
    return (f, t, build_tree(X[left], y[left], depth + 1, n_features),
            build_tree(X[~left], y[~left], depth + 1, n_features))


def predict_tree(tree, row):
    while isinstance(tree, tuple):
        f, t, left, right = tree
        tree = left if row[f] <= t else right
    return tree


df = pd.read_csv("heart.csv")
print(df.info())
print(df.isnull().sum())
df = df.dropna()

X = df.drop("target", axis=1).values.astype(float)
y = df["target"].values.astype(int)

np.random.seed(42)
index = np.random.permutation(len(X))
split = int(0.8 * len(X))
X_train, X_test = X[index[:split]], X[index[split:]]
y_train, y_test = y[index[:split]], y[index[split:]]

np.random.seed(42)
trees = []
for i in range(10):
    sample = np.random.randint(0, len(X_train), len(X_train))
    trees.append(build_tree(X_train[sample], y_train[sample], 0, int(np.sqrt(X_train.shape[1])) + 1))

votes = np.array([[predict_tree(tree, row) for row in X_test] for tree in trees])
y_pred = np.array([np.bincount(votes[:, j]).argmax() for j in range(votes.shape[1])])

tp = np.sum((y_test == 1) & (y_pred == 1))
tn = np.sum((y_test == 0) & (y_pred == 0))
fp = np.sum((y_test == 0) & (y_pred == 1))
fn = np.sum((y_test == 1) & (y_pred == 0))

accuracy = (tp + tn) / len(y_test)
precision = tp / (tp + fp)
recall = tp / (tp + fn)
f1 = 2 * precision * recall / (precision + recall)

print("Accuracy :", accuracy)
print("Precision:", precision)
print("Recall   :", recall)
print("F1-Score :", f1)
print("Confusion Matrix:")
print(np.array([[tn, fp], [fn, tp]]))
```

Slip 20

Q1 [10 Marks]

Create a DataFrame with Age, Salary, Experience, Working_Hours and Performance_Score. Apply Standardization and display the transformed data.

```
import pandas as pd

df = pd.DataFrame({
    "Age": [25, 32, 41, 28, 36, 45],
    "Salary": [30000, 45000, 70000, 38000, 52000, 85000],
    "Experience": [2, 8, 16, 4, 10, 20],
    "Working_Hours": [8, 9, 8, 10, 9, 8],
    "Performance_Score": [6.5, 7.8, 8.9, 7.0, 8.2, 9.3]
})
print("Original data:")
print(df)

standardized = (df - df.mean()) / df.std(ddof=0)

print("Standardized data:")
print(standardized)
```

Q2 [20 Marks]

Create a dataset of books (Book Title, Author, Publication Year, Average Rating, Number of Ratings). (a) Create a Venn Diagram comparing the authors of books published in two time periods. (b) Generate a Word Cloud from the book titles.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib_venn import venn2

books = pd.DataFrame({
    "Book_Title": ["Harry Potter and the Philosophers Stone", "The Da Vinci Code",
                   "The Alchemist", "Harry Potter and the Half Blood Prince",
                   "The Kite Runner", "The Lost Symbol", "A Thousand Splendid Suns",
                   "The Hunger Games"],
    "Author": ["J.K. Rowling", "Dan Brown", "Paulo Coelho", "J.K. Rowling",
               "Khaled Hosseini", "Dan Brown", "Khaled Hosseini", "Suzanne Collins"],
    "Publication_Year": [1997, 2003, 1988, 2005, 2003, 2009, 2007, 2008],
    "Average_Rating": [4.47, 3.79, 3.82, 4.57, 4.30, 3.66, 4.37, 4.33],
    "Number_of_Ratings": [6000000, 1500000, 2300000, 2400000, 2700000, 1100000, 1100000, 7000000]
})
print(books)

period1 = set(books[books["Publication_Year"] < 2005]["Author"])
period2 = set(books[books["Publication_Year"] >= 2005]["Author"])

venn2([period1, period2], set_labels=("Before 2005", "2005 and later"))
plt.title("Authors in Two Time Periods")
plt.show()

stop_words = ["the", "of", "and", "a"]
words = " ".join(books["Book_Title"]).lower().split()
words = [w for w in words if w not in stop_words]
frequency = pd.Series(words).value_counts()
print(frequency)

np.random.seed(1)
plt.figure(figsize=(8, 5))
for i, (word, count) in enumerate(frequency.items()):
    x = 0.15 + (i % 4) * 0.23
    y = 0.9 - int(i / 4) * 0.2
    plt.text(x, y, word, fontsize=12 + 10 * count, color=np.random.rand(3), ha="center")
plt.axis("off")
plt.title("Word Cloud of Book Titles")
plt.show()
```

Q2 OR [20 Marks]

Apply Apriori to the student activity dataset to find frequent itemsets and association rules. Repeat with minimum support 0.2 and 0.3.

```
import pandas as pd
from itertools import combinations

def apriori(transactions, min_support):
    n = len(transactions)
    items = sorted(set(i for t in transactions for i in t))
    frequent = {}
    current = [(i,) for i in items]
    while current:
        found = {}
        for itemset in current:
            count = sum(1 for t in transactions if set(itemset).issubset(t))
            if count / n >= min_support:
                found[itemset] = count / n
        frequent.update(found)
        keys = list(found)
        current = set()
        for a in range(len(keys)):
            for b in range(a + 1, len(keys)):
                union = tuple(sorted(set(keys[a]) | set(keys[b])))
                if len(union) == len(keys[a]) + 1:
                    current.add(union)
        current = list(current)
    return frequent


def association_rules(frequent, min_confidence):
    rules = []
    for itemset, support in frequent.items():
        if len(itemset) < 2:
            continue
        for r in range(1, len(itemset)):
            for antecedent in combinations(itemset, r):
                consequent = tuple(i for i in itemset if i not in antecedent)
                confidence = support / frequent[antecedent]
                lift = confidence / frequent[consequent]
                if confidence >= min_confidence:
                    rules.append([antecedent, consequent, support, confidence, lift])
    return pd.DataFrame(rules, columns=["antecedent", "consequent", "support", "confidence", "lift"])


transactions = [
    ["Library", "Seminar"],
    ["Workshop", "Project"],
    ["Library", "Assignment"],
    ["Seminar", "Workshop"],
    ["Assignment", "Seminar"],
    ["Workshop", "Assignment"],
    ["Project", "Library"],
    ["Library", "Seminar", "Project"],
    ["Library", "Seminar", "Workshop"]
]

for support in [0.2, 0.3]:
    print("Minimum support =", support)
    frequent = apriori(transactions, support)
    print(pd.DataFrame(list(frequent.items()), columns=["itemsets", "support"]))
    print(association_rules(frequent, 0.5))
```

Slip 22

Q1 [10 Marks]

Create a DataFrame with City, House_Type, House_Area, Number_of_Rooms and Monthly_Rent. Apply the Normalization technique to the numerical attributes and display the normalized data.

```
import pandas as pd

df = pd.DataFrame({
    "City": ["Nashik", "Pune", "Mumbai", "Nagpur", "Pune", "Mumbai"],
    "House_Type": ["Flat", "Villa", "Flat", "Row House", "Flat", "Studio"],
    "House_Area": [850, 2200, 650, 1400, 1000, 400],
    "Number_of_Rooms": [2, 5, 2, 3, 3, 1],
    "Monthly_Rent": [9000, 45000, 32000, 15000, 18000, 22000]
})
print("Original data:")
print(df)

cols = ["House_Area", "Number_of_Rooms", "Monthly_Rent"]
df[cols] = df[cols].div(((df[cols] ** 2).sum(axis=1)) ** 0.5, axis=0)

print("Normalized data:")
print(df)
```

Q2 [20 Marks]

Generate a DataFrame of Netflix and Amazon Prime customers. Create a Venn diagram and a Pie chart for only Netflix, only Amazon Prime and both, with labels, colors and titles.

```
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib_venn import venn2

df = pd.DataFrame({
    "Customer": ["Amit", "Neha", "Rohan", "Sara", "Karan", "Priya", "Vikas", "Anita"],
    "Netflix": [1, 1, 0, 1, 0, 1, 1, 0],
    "Prime": [0, 1, 1, 1, 1, 0, 1, 1]
})
print(df)

netflix = set(df[df["Netflix"] == 1]["Customer"])
prime = set(df[df["Prime"] == 1]["Customer"])

only_netflix = len(netflix - prime)
only_prime = len(prime - netflix)
both = len(netflix & prime)

venn2([netflix, prime], set_labels=("Netflix", "Amazon Prime"), set_colors=("red", "blue"))
plt.title("Netflix and Amazon Prime Customers")
plt.show()

plt.pie([only_netflix, only_prime, both], labels=["Only Netflix", "Only Prime", "Both"],
        colors=["red", "blue", "yellow"], autopct="%1.1f%%")
plt.title("Netflix and Amazon Prime Customers")
plt.show()
```

Q2 OR [20 Marks]

Apply K-Means Clustering to a student dataset with Attendance and Marks. Preprocess, use K = 3, and visualize the clusters with their centroids on a scatter plot.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

def kmeans(X, k, iterations=100):
    np.random.seed(42)
    centers = X[np.random.choice(len(X), k, replace=False)]
    for i in range(iterations):
        distances = np.sqrt(((X[:, None, :] - centers[None, :, :]) ** 2).sum(axis=2))
        labels = distances.argmin(axis=1)
        new_centers = np.array([X[labels == j].mean(axis=0) if np.any(labels == j)
                                else centers[j] for j in range(k)])
        if np.allclose(new_centers, centers):
            break
```

```
        centers = new_centers
    return labels, centers


students = pd.DataFrame({
    "Attendance": [95, 92, 90, 88, 94, 75, 72, 78, 70, 74, 55, 50, 58, 52, 60],
    "Marks": [90, 85, 88, 82, 92, 65, 60, 68, 58, 62, 40, 35, 45, 38, 42]
})
print(students.isnull().sum())

X = students.values.astype(float)
X = (X - X.mean(axis=0)) / X.std(axis=0)

labels, centers = kmeans(X, 3)
students["Cluster"] = labels
print(students)

plt.scatter(X[:, 0], X[:, 1], c=labels)
plt.scatter(centers[:, 0], centers[:, 1], color="red", marker="X", s=200, label="Centroids")
plt.xlabel("Attendance (scaled)")
plt.ylabel("Marks (scaled)")
plt.title("K-Means Clustering (K = 3)")
plt.legend()
plt.show()
```

Slip 23

Q1 [10 Marks]

Create a DataFrame of products (Product_name, Price, Rating, Number_of_Sales). Generate a Dendrogram to identify groups of similar products, with a title and labels.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

df = pd.DataFrame({
    "Product_name": ["Laptop", "Phone", "Tablet", "Headphones", "Smartwatch", "Camera", "Speaker", "Mouse"],
    "Price": [55000, 30000, 25000, 3000, 8000, 40000, 5000, 800],
    "Rating": [4.5, 4.4, 4.2, 4.0, 4.1, 4.6, 3.9, 4.0],
    "Number_of_Sales": [1200, 5000, 1800, 6000, 3500, 700, 4200, 7000]
})
print(df)

X = df[["Price", "Rating", "Number_of_Sales"]].values.astype(float)
X = (X - X.mean(axis=0)) / X.std(axis=0)
n = len(X)
dist = np.sqrt(((X[:, None, :] - X[None, :, :]) ** 2).sum(axis=2))

nodes = {i: i for i in range(n)}
members = {i: [i] for i in range(n)}
next_id = n
while len(nodes) > 1:
    best = None
    ids = list(nodes)
    for a in range(len(ids)):
        for b in range(a + 1, len(ids)):
            d = dist[np.ix_(members[ids[a]], members[ids[b]])].mean()
            if best is None or d < best[0]:
                best = (d, ids[a], ids[b])
    d, a, b = best
    nodes[next_id] = (nodes.pop(a), nodes.pop(b), d)
    members[next_id] = members.pop(a) + members.pop(b)
    next_id += 1
root = nodes[next_id - 1]


def leaves(node):
    if isinstance(node, int):
        return [node]
    return leaves(node[0]) + leaves(node[1])


order = leaves(root)
position = {leaf: i for i, leaf in enumerate(order)}


def draw(node):
    if isinstance(node, int):
        return position[node], 0
    xl, hl = draw(node[0])
    xr, hr = draw(node[1])
    plt.plot([xl, xl, xr, xr], [hl, node[2], node[2], hr], color="blue")
    return (xl + xr) / 2, node[2]


draw(root)
plt.xticks(range(n), df["Product_name"].values[order], rotation=90)
plt.title("Dendrogram of Products")
plt.xlabel("Products")
plt.ylabel("Distance")
plt.tight_layout()
plt.show()
```

Q2 [20 Marks]

Create a DataFrame with Temperature, Humidity, Wind Speed, Pressure and Rainfall. Apply (a) Min-Max Scaling, (b) Standardization and (c) Normalization and display the transformed data.

```
import pandas as pd

df = pd.DataFrame({
```

```
    "Temperature": [30.5, 28.0, 35.2, 22.1, 25.6, 33.0],
    "Humidity": [65, 70, 50, 85, 78, 55],
    "WindSpeed": [12, 15, 8, 20, 18, 9],
    "Pressure": [1010, 1008, 1005, 1015, 1012, 1006],
    "Rainfall": [5, 12, 0, 30, 20, 1]
})
print("Original data:")
print(df)

minmax = (df - df.min()) / (df.max() - df.min())
print("Min-Max Scaling:")
print(minmax)

standard = (df - df.mean()) / df.std(ddof=0)
print("Standardization:")
print(standard)

normal = df.div(((df ** 2).sum(axis=1)) ** 0.5, axis=0)
print("Normalization:")
print(normal)
```

Q2 OR [20 Marks]

Implement a KNN classifier on the Student Grading dataset to classify students into performance categories. Preprocess, train, test and evaluate with Accuracy, Precision, Recall, F1-Score and Confusion Matrix.

```
import numpy as np
import pandas as pd

def knn_predict(X_train, y_train, X_test, k=5):
    predictions = []
    for row in X_test:
        distances = np.sqrt(((X_train - row) ** 2).sum(axis=1))
        nearest = y_train[np.argsort(distances)[:k]]
        values, counts = np.unique(nearest, return_counts=True)
        predictions.append(values[counts.argmax()])
    return np.array(predictions)


df = pd.read_csv("Students_Grading_Dataset.csv")
print(df.isnull().sum())

df = df.drop(["Student_ID", "First_Name", "Last_Name", "Email", "Total_Score"], axis=1)
df = df.dropna()

for col in df.select_dtypes(exclude="number").columns:
    df[col] = df[col].astype("category").cat.codes

X = df.drop("Grade", axis=1).values.astype(float)
y = df["Grade"].values.astype(int)

np.random.seed(42)
index = np.random.permutation(len(X))
split = int(0.8 * len(X))
X_train, X_test = X[index[:split]], X[index[split:]]
y_train, y_test = y[index[:split]], y[index[split:]]

mean = X_train.mean(axis=0)
std = X_train.std(axis=0) + 1e-9
X_train = (X_train - mean) / std
X_test = (X_test - mean) / std

y_pred = knn_predict(X_train, y_train, X_test, k=5)

classes = np.unique(y_test)
precision = []
recall = []
for c in classes:
    tp = np.sum((y_test == c) & (y_pred == c))
    fp = np.sum((y_test != c) & (y_pred == c))
    fn = np.sum((y_test == c) & (y_pred != c))
    precision.append(tp / (tp + fp) if tp + fp > 0 else 0)
    recall.append(tp / (tp + fn) if tp + fn > 0 else 0)

precision = np.mean(precision)
recall = np.mean(recall)
f1 = 2 * precision * recall / (precision + recall)

print("Accuracy :", np.mean(y_test == y_pred))
```

```
print("Precision:", precision)
print("Recall   :", recall)
print("F1-Score :", f1)
print("Confusion Matrix:")
print(pd.crosstab(y_test, y_pred, rownames=["Actual"], colnames=["Predicted"]))
```

Note: Grade is the target. Total_Score is dropped because the grade is calculated from it. KNN (k = 5) is written with numpy. Precision, Recall and F1 are macro averages.

Slip 25

Q1 [10 Marks]

Bar Chart of students enrolled: courses = [BCA, BBA, B.Com, B.Sc, BA], students = [60, 45, 75, 50, 35]. Use different bar colors, add a title and label both axes.

```
import matplotlib.pyplot as plt

courses = ["BCA", "BBA", "B.Com", "B.Sc", "BA"]
students = [60, 45, 75, 50, 35]
colors = ["blue", "orange", "green", "red", "purple"]

plt.bar(courses, students, color=colors)
plt.title("Number of Students Enrolled in Each Course")
plt.xlabel("Courses")
plt.ylabel("Number of Students")
plt.show()
```

Q2 [20 Marks]

Create two DataFrames of mobile phones (DataFrame 1: Brand, Model, Price, RAM; DataFrame 2: Model, Storage, Battery_Capacity, Operating_System). Perform data integration on Model, Label Encoding on Brand and One-Hot Encoding on Operating_System. Display the original, merged and transformed DataFrames.

```
import pandas as pd

df1 = pd.DataFrame({
    "Brand": ["Samsung", "Apple", "OnePlus", "Xiaomi"],
    "Model": ["Galaxy S23", "iPhone 14", "OnePlus 11", "Redmi Note 12"],
    "Price": [74999, 79900, 56999, 17999],
    "RAM": [8, 6, 12, 6]
})
df2 = pd.DataFrame({
    "Model": ["Galaxy S23", "iPhone 14", "OnePlus 11", "Redmi Note 12"],
    "Storage": [256, 128, 256, 128],
    "Battery_Capacity": [3900, 3279, 5000, 5000],
    "Operating_System": ["Android", "iOS", "Android", "Android"]
})
print(df1)
print(df2)

merged = pd.merge(df1, df2, on="Model")
print("Merged DataFrame:")
print(merged)

merged["Brand"] = merged["Brand"].astype("category").cat.codes
transformed = pd.get_dummies(merged, columns=["Operating_System"])
print("Transformed DataFrame:")
print(transformed)
```

Q2 OR [20 Marks]

Market basket analysis on the Groceries dataset using Apriori: explore, handle missing values, convert to a numeric format, generate frequent itemsets and rules for minimum support 0.3 and 0.4.

```
import pandas as pd
from itertools import combinations

def apriori(transactions, min_support):
    n = len(transactions)
    items = sorted(set(i for t in transactions for i in t))
    frequent = {}
    current = [(i,) for i in items]
    while current:
        found = {}
        for itemset in current:
            count = sum(1 for t in transactions if set(itemset).issubset(t))
            if count / n >= min_support:
                found[itemset] = count / n
        frequent.update(found)
        keys = list(found)
        current = set()
        for a in range(len(keys)):
            for b in range(a + 1, len(keys)):
                union = tuple(sorted(set(keys[a]) | set(keys[b])))
```

```
                if len(union) == len(keys[a]) + 1:
                    current.add(union)
        current = list(current)
    return frequent


def association_rules(frequent, min_confidence):
    rules = []
    for itemset, support in frequent.items():
        if len(itemset) < 2:
            continue
        for r in range(1, len(itemset)):
            for antecedent in combinations(itemset, r):
                consequent = tuple(i for i in itemset if i not in antecedent)
                confidence = support / frequent[antecedent]
                lift = confidence / frequent[consequent]
                if confidence >= min_confidence:
                    rules.append([antecedent, consequent, support, confidence, lift])
    return pd.DataFrame(rules, columns=["antecedent", "consequent", "support", "confidence", "lift"])


df = pd.DataFrame({"Items": [
    "Basmati Rice, Toor Dal, Turmeric Powder, Cooking Oil",
    "Atta, Sugar, Tea, Milk",
    "Rice, Moong Dal, Salt, Cooking Oil",
    "Wheat Flour, Chickpeas, Turmeric Powder, Cumin Seeds",
    "Poha, Peanuts, Onions, Cooking Oil",
    "Basmati Rice, Paneer, Garam Masala, Tomatoes",
    "Atta, Potatoes, Onions, Cooking Oil"]})
print(df)
print(df.isnull().sum())
df = df.dropna()

transactions = [row.split(", ") for row in df["Items"]]

for support in [0.3, 0.4]:
    print("Minimum support =", support)
    frequent = apriori(transactions, support)
    print(pd.DataFrame(list(frequent.items()), columns=["itemsets", "support"]))
    print(association_rules(frequent, 0.5))
```

Note: With these 7 transactions only Cooking Oil is frequent at 0.3 and 0.4, so the rules table will be empty. That is the correct result. Apriori is written with itertools and pandas.